CSC-239
Java Programming:
Chapter 7: Single-dimensional Arrays (Part 2)

September 29, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Announcement: Quiz 2

- Quiz 2 will begin on Monday, October 6.
  - Online (Moodle) Quiz.
  - The Quiz will open on Monday at 9:00 p.m. and remain open for approximately one week.
  - Each student will have two hours to complete the Quiz, from the time that they begin.
- The material covered on Quiz 2 will be
  - The lectures of September 10 through 29.
  - Chapters 4 through 7.
  - The quiz will include True / False, Multiple-Choice questions.

Reminder: Submitting Files to *Moodle*

- When you submit your source code (*.java) files to *Moodle*, you MUST follow the **Submit Assignment to *Moodle*** procedure *completely*: be sure that the final Submission Status is:

  "Submitted for grading".



- This procedure is described in detail in the **Submit_Assignment_to_Moodle.pdf** document, which is available on *Moodle*, under the "**Handouts**" topic.

Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

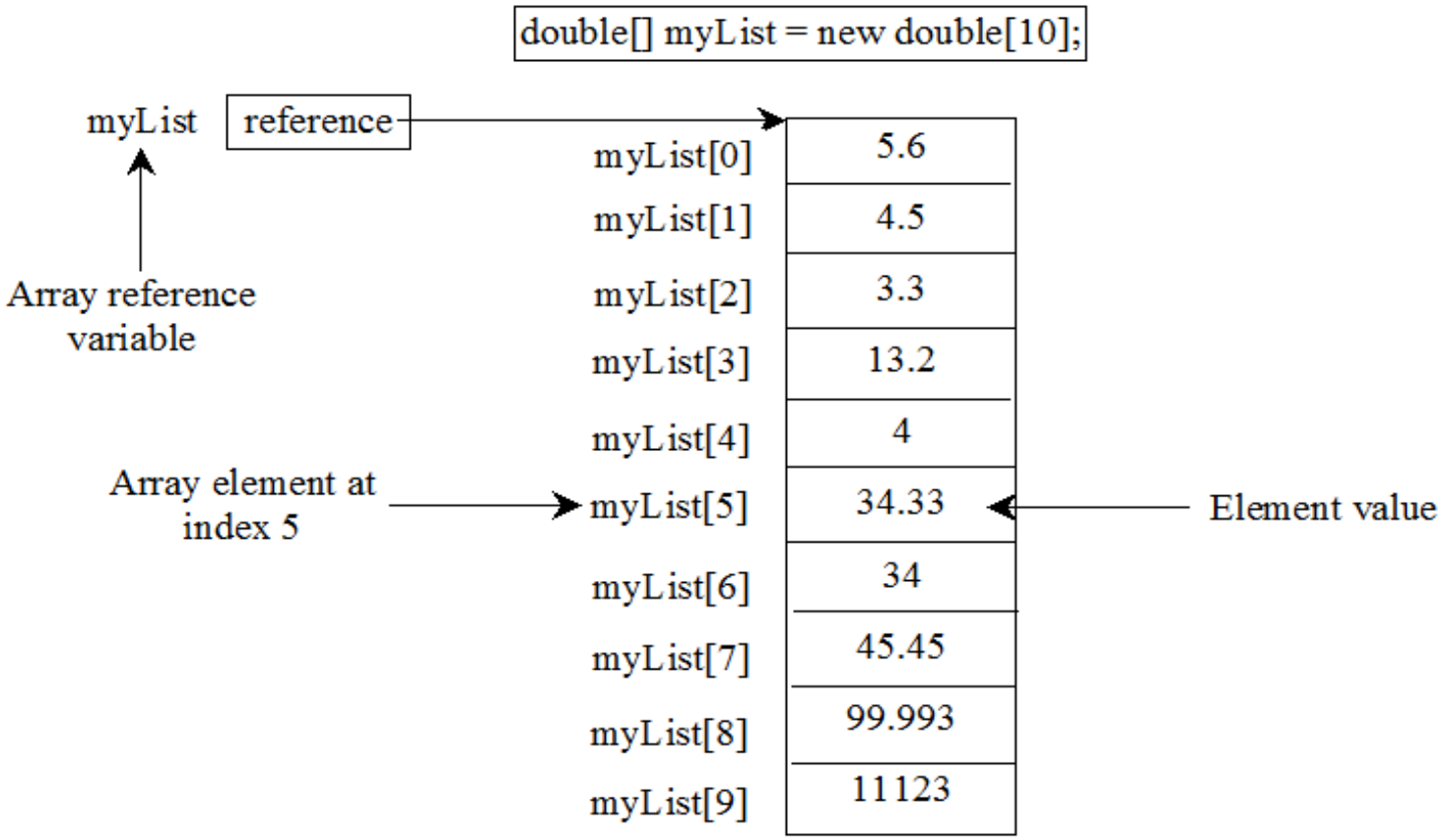
- *Grader Than* Support E-mail:

  support@graderthan.com

Previous Lecture: Arrays

We introduced the concept of an array: a container object that holds a number of values of a single type.

[http://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html]



Searching Arrays

Searching is the process of looking for a specific value in an array.

We are considering two specific algorithms for searching an array:

- Linear Search ("Sequential Search").
- Binary Search

In this discussion, the data value that we are looking for is called the search ***key***.

The search ***key*** could also be thought of as the ***search target*** (the value that we are looking for).

Linear Search

The linear search approach compares the key element, **`key`**, *sequentially* with each element in the array **`list`**.

The method continues until either

- The **`key`** value is found in the array, or
- The end of the array is reached.

Linear Search (Sequential Search)

This example searches for an occurrence of **`key`** in the integer array **`list`**. The method searches for the value **`key`** in the array **`list`**, returning the index of the **`key`** value, if it is found, or returns **`-1`** if the **`key`** value is not found:

```
public static int linearSearch(int[] list,
        int key) {
    for (int i = 0; i < list.length; i++) {
      if (key == list[i])
        return i;
    } // (end for loop)
    return -1;
}
```

animation

Linear Search Animation

The URL below links to an interactive demonstration of the Linear Search process:

https://liveexample.pearsoncmg.com/dsanimation/LinearSearcheBook.html

The website includes instructions for the user to select the desired size of the array and the particular **key** (search target) value. The user repeatedly clicks on the "Next" button as the search algorithm executes in a step-by-step manner.

Binary Search (1)

- For binary search to work, the elements in the array must already be ordered (sorted).
- For this discussion, assume that the data in the array is in *ascending* order.

  For example:

```
2 4 7 10 11 45 50 59 60 66 69 70 79
```

- The binary search first compares the **key** (search target) with the element in the "middle" of the array.

Binary Search (2)

Consider the following three cases:

- If the **key** is equal to the data value at the "middle" index, the search ends with a match.
- If the **key** is less than the data value at the "middle" index, you only need to search the data in the *first half* of the array.
- If the **key** is greater than the data value at the middle index, you only need to search the key in the *second half* of the array.

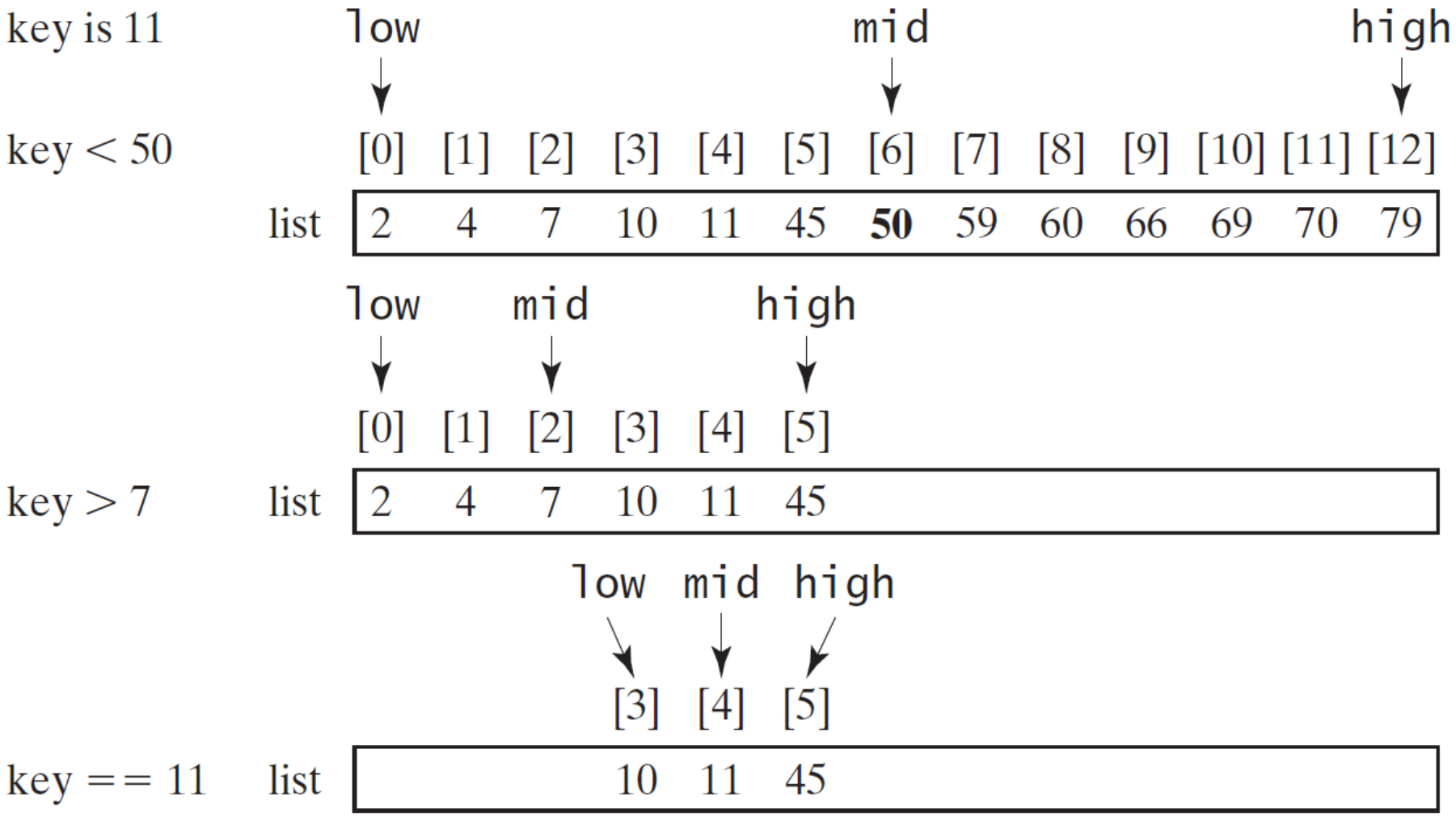
animation

Binary Search Animation

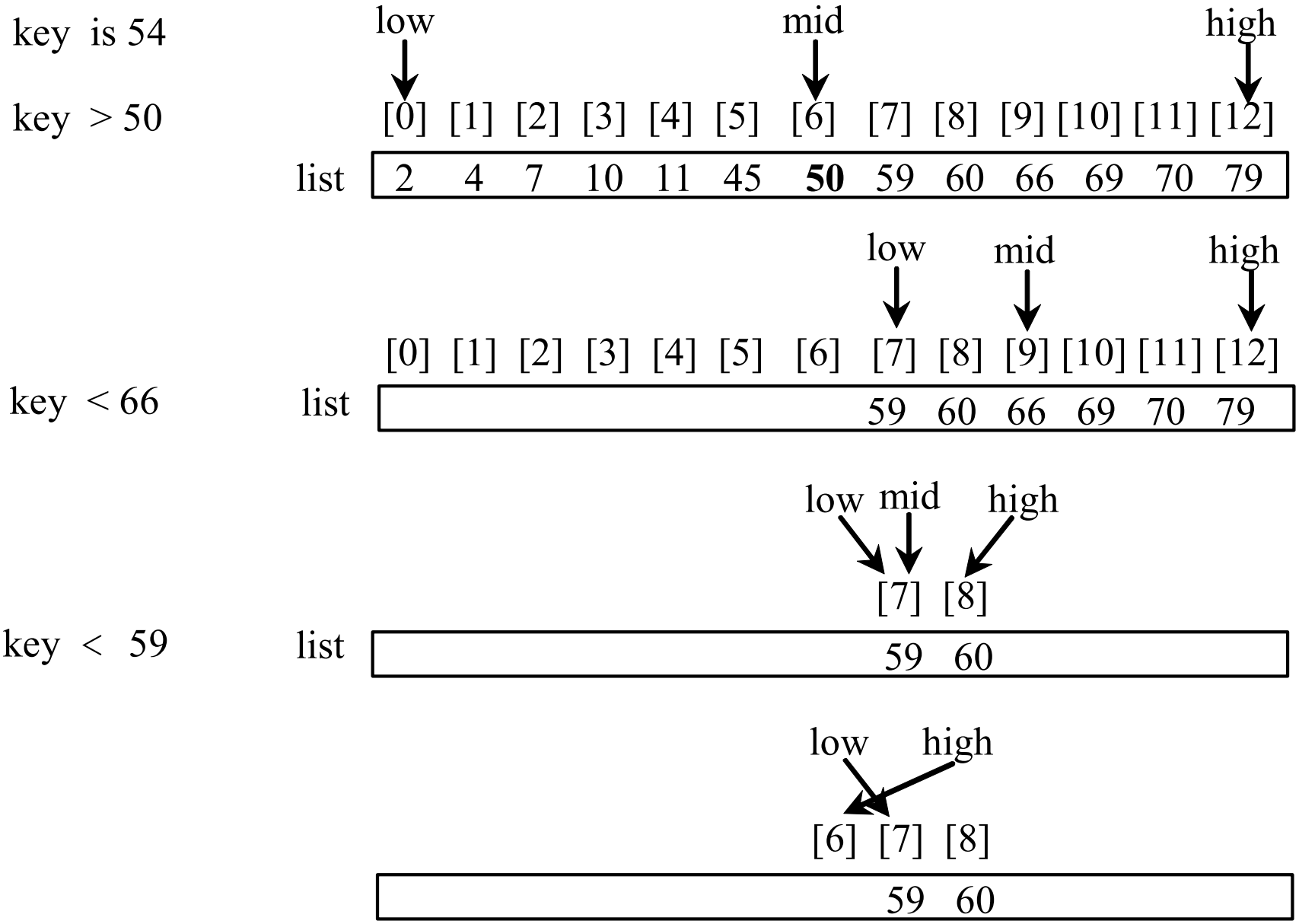
The URL below links to an interactive demonstration of the Binary Search process:

https://liveexample.pearsoncmg.com/dsanimation/BinarySearcheBook.html

Binary Search Example: ***match found***



Binary Search Example: ***not found***



Binary Search *return value*

- The **binarySearch** method returns the index of the element in the list that matches the search **key** (if the **key** value *is* contained in the list.
- If the **key** value *is not* contained in the array, then the method returns

```
-insertion point - 1
```

- The "insertion point" is the array index where the **key** value would be inserted into the list.

Code for the Binary Search

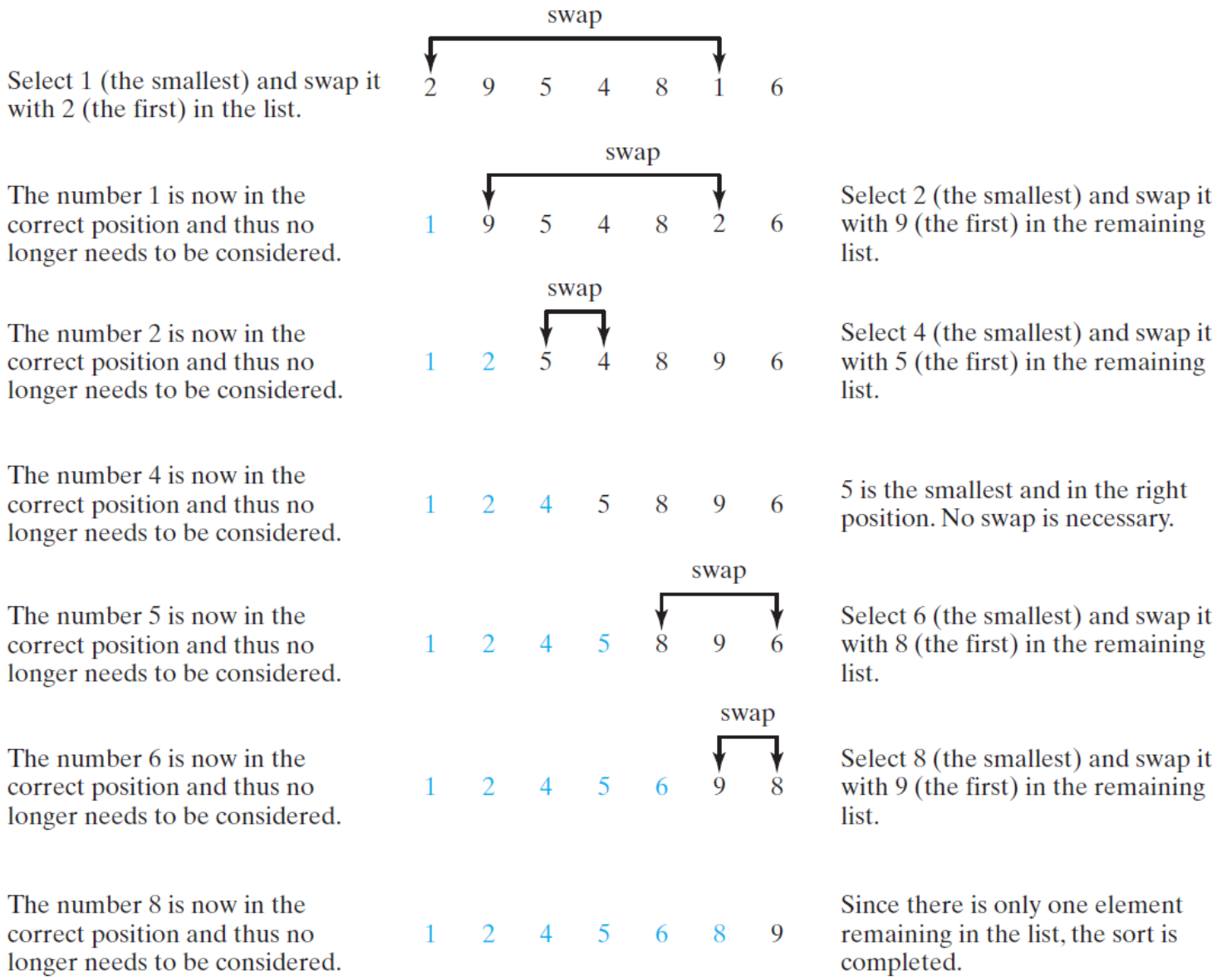
```
// Binary Search (data must be sorted)
public static int binarySearch(int[] list, int key){
  int low = 0;
  int high = list.length - 1;

  while (high >= low) {
    int mid = (low + high) / 2;
    if (key < list[mid])
      high = mid - 1;
    else if (key == list[mid])
      return mid;                    // FOUND
    else
      low = mid + 1;
  }
  return -1 - low;                   // NOT FOUND
}
```

Sorting Arrays

- There are many different algorithms for sorting arrays.
- We are looking at a simple, intuitive sorting algorithm: *selection sort:*
- Selection sort finds the *smallest* value in the list and places it first. It then finds the next smallest value and places it second, and so on until the list contains only a single remaining number.

Selection Sort



animation

Selection Sort Animation

The URL below links to an interactive demonstration of the Selection Sort process:

https://liveexample.pearsoncmg.com/dsanimation/SelectionSortNew.html

The `java.util.Arrays` class

This pre-defined class contains various utility methods for working with arrays.

These methods include:

- Binary search
- Sort
- toString

The `Arrays.sort` Method

- The **java.util.Arrays** class provides several overloaded sort methods for sorting an array of various data types.
- The following example sorts an array of numbers:

```
double[] numbers={6.0, 4.4, 1.9, 2.9,
                  3.4, 3.5};
java.util.Arrays.sort(numbers);
```

- The following example sorts an array of characters:

```
char[] chars={'a', 'A', '4', 'F', 'D', 'P'};
java.util.Arrays.sort(chars);
```

The `Arrays.toString` Method

The **Arrays.toString** method can be used to return a string representation for the list.

Command-Line Parameters

- You may have noticed that the **main** method of a Java program takes an array of **String** objects as its parameters:

```
class TestMain {
    public static void main(String[] args){
        ...
    }
}
```

- We could execute this program from the command-line:

```
java TestMain arg0 arg1 arg2 ... argn
```

Processing Command-Line Parameters

In the main method, get array elements

```
args[0], args[1], ..., args[n]
```

contain the text arguments that the user typed on the command line:

```
arg0, arg1, ..., argn
```